

Linear co-design problems: Composition theory, solving algorithms, and approximation

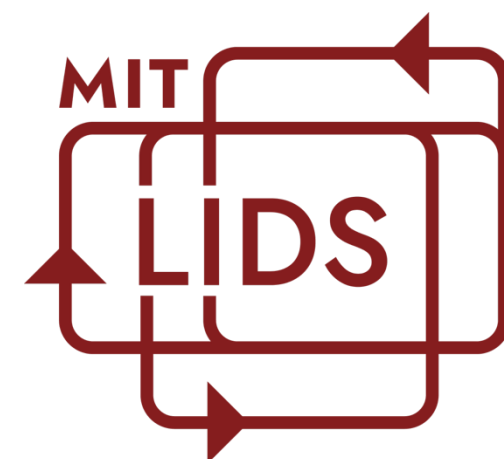
Control Architecture Theory meeting

April 1, 2026

Yubo Cai, Yujun Huang, Meshal Alharbi, Gioele Zardini

LIDS, IDSS, CEE

Massachusetts Institute of Technology, Cambridge (MA), USA



Need new tools to model and solve complex systems design optimization problems

- Societal impact of new technologies depends on their joint design with existing infrastructure and systems



Intermodal mobility networks



Networks of ships

Need new tools to model and solve complex systems design optimization problems

- Societal impact of new technologies depends on their joint design with existing infrastructure and systems



Intermodal mobility networks



Networks of ships

Example - Autonomy: Heaven or hell?

30% of the cars would be enough

First- and last-mile mobility could make public transit more convenient and attractive

More affordable, sustainable



Your Uber Car Creates Congestion. Should You Pay a Fee to Ride? (New York Times)



Single components are slowly well understood, but we still lack a (formal and practical) theory for the task-driven co-design of complex systems

Autonomy as the frontier of complexity for the co-design of complex systems

A fleet of autonomous agents



=

actuation hardware software behavior coordination
sensing localization planning liability
computation control interaction learning
perception mapping regulations
energetics communication infrastructure

OMG!

So many components (hardware, software, ...),
objectives and choices to make!
Nobody understands the whole thing!

We forget why we made choices, and we are afraid to
make changes (high failure cost).
We need faster design cycles, nimbler execution.

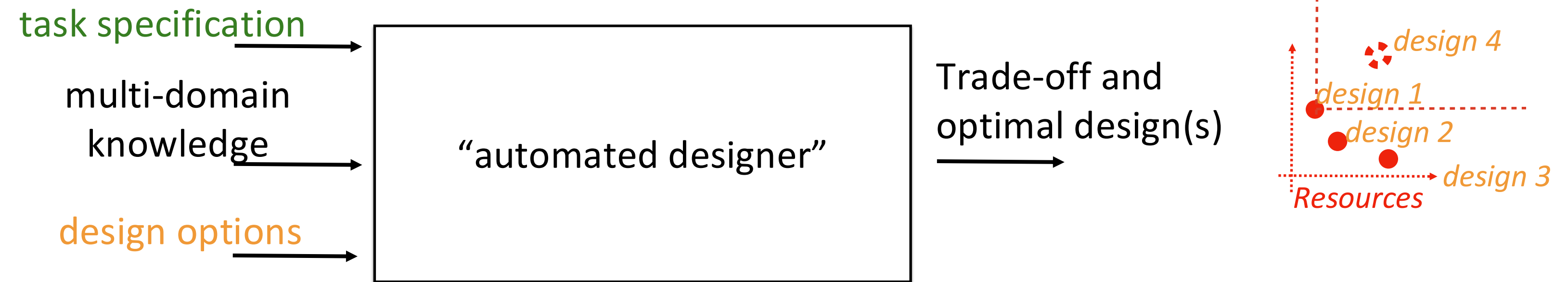
Engineer in the
21st century



“My dear, it’s simple: you lack
a *practical* theory of co-design!”
Formal,
Quantitative
Intellectually tractable

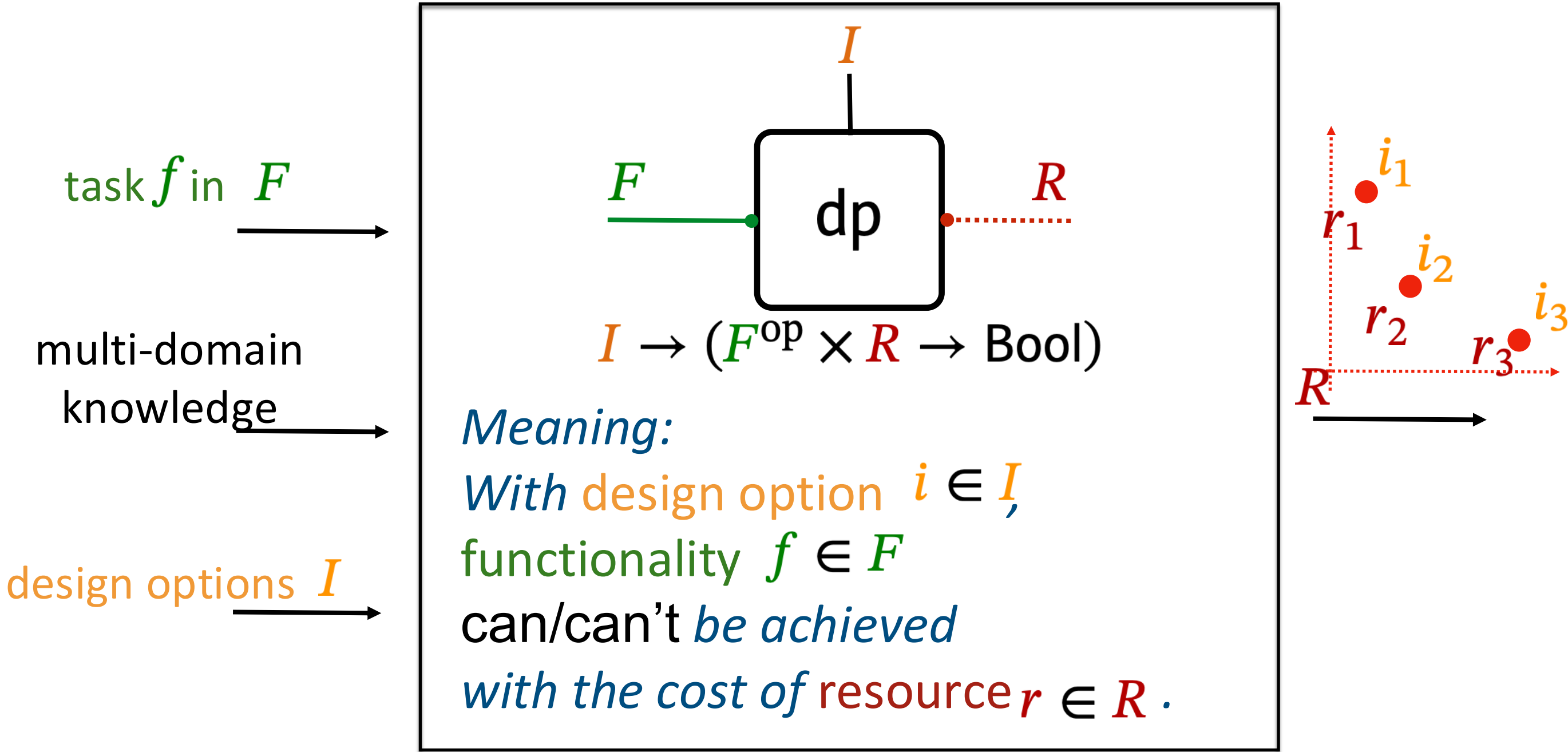
The vision of automated co-design

minimize
resources usage
subject to
functionality constraints
with
(combination of) available
design choices



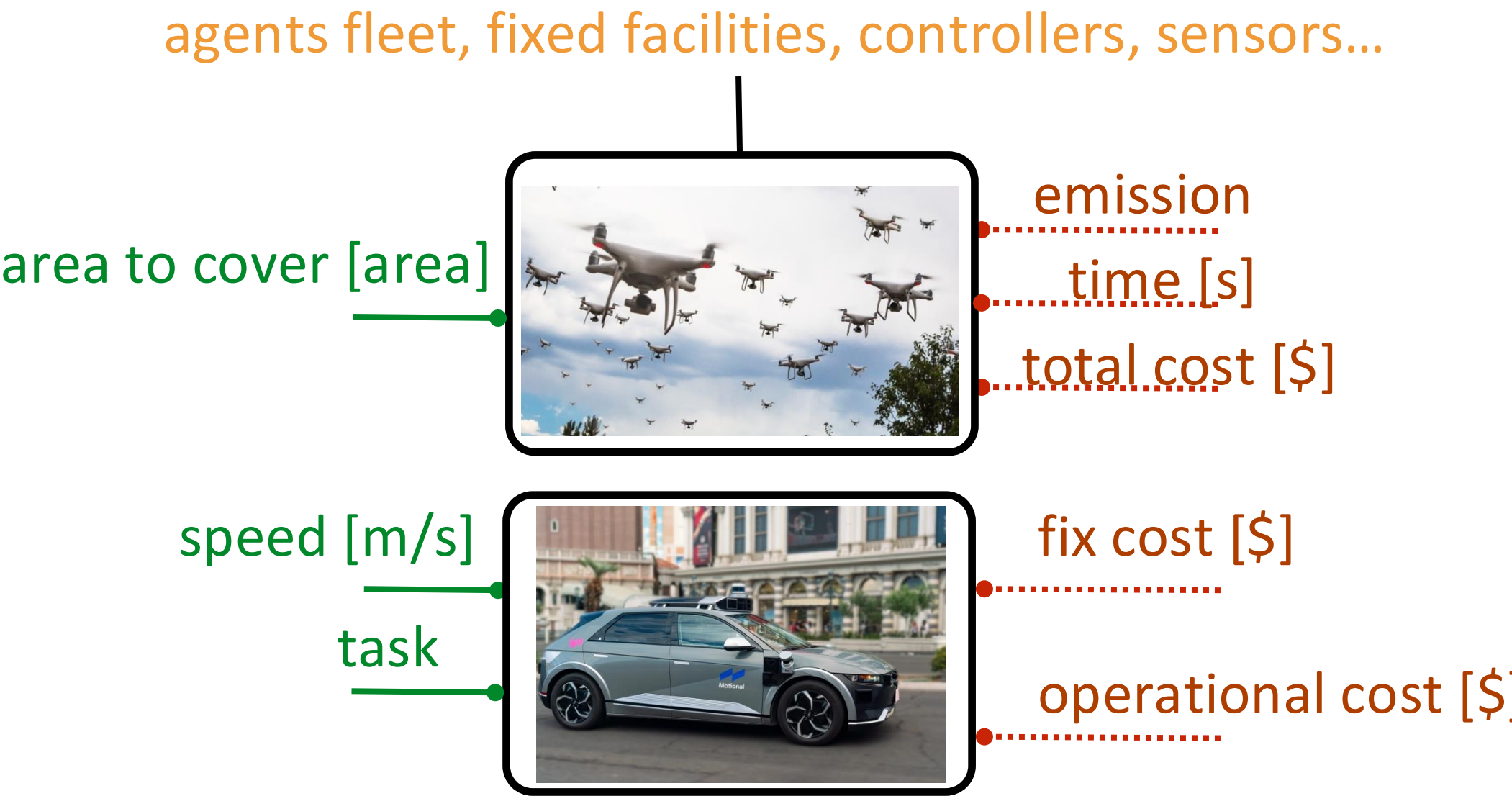
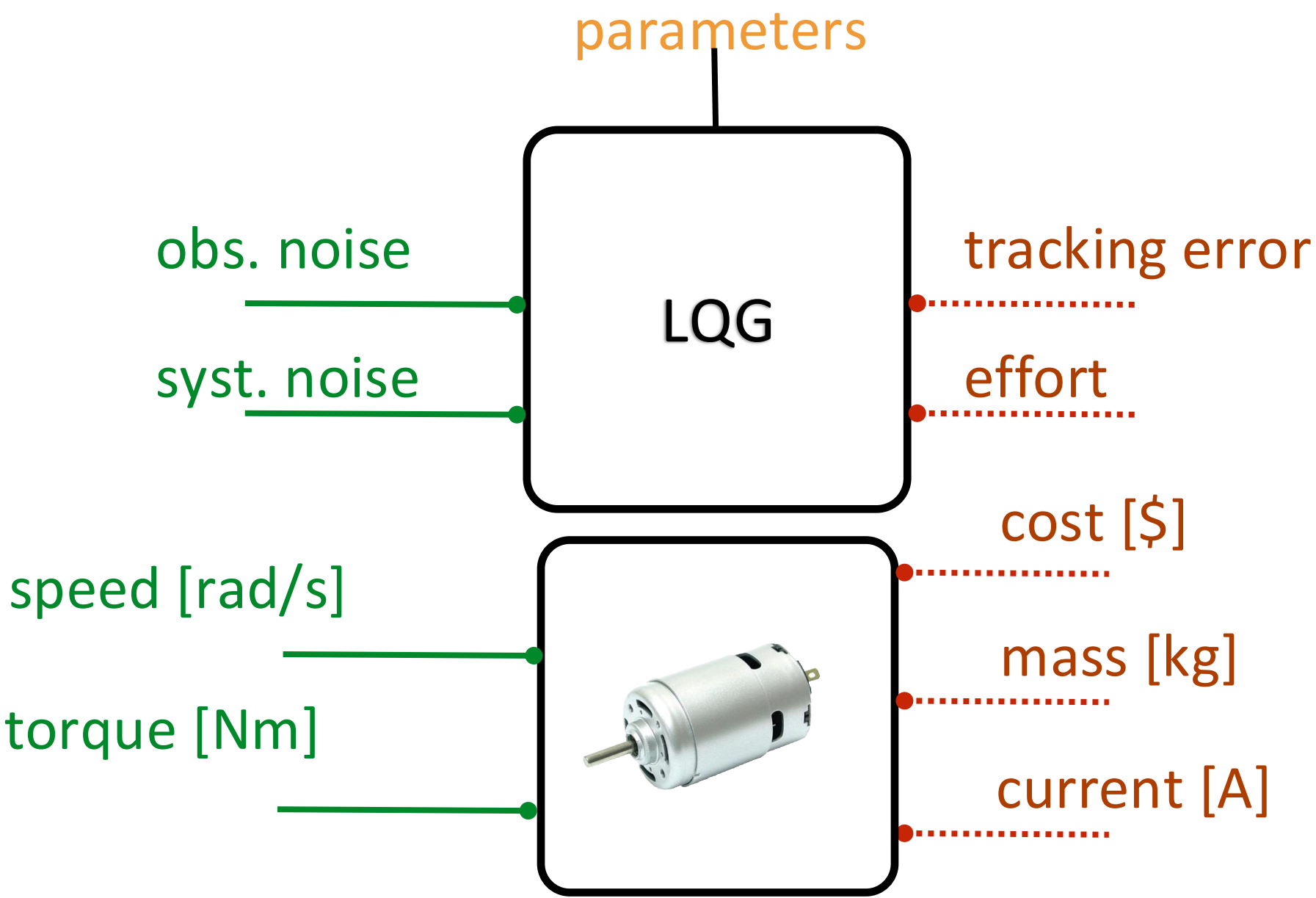
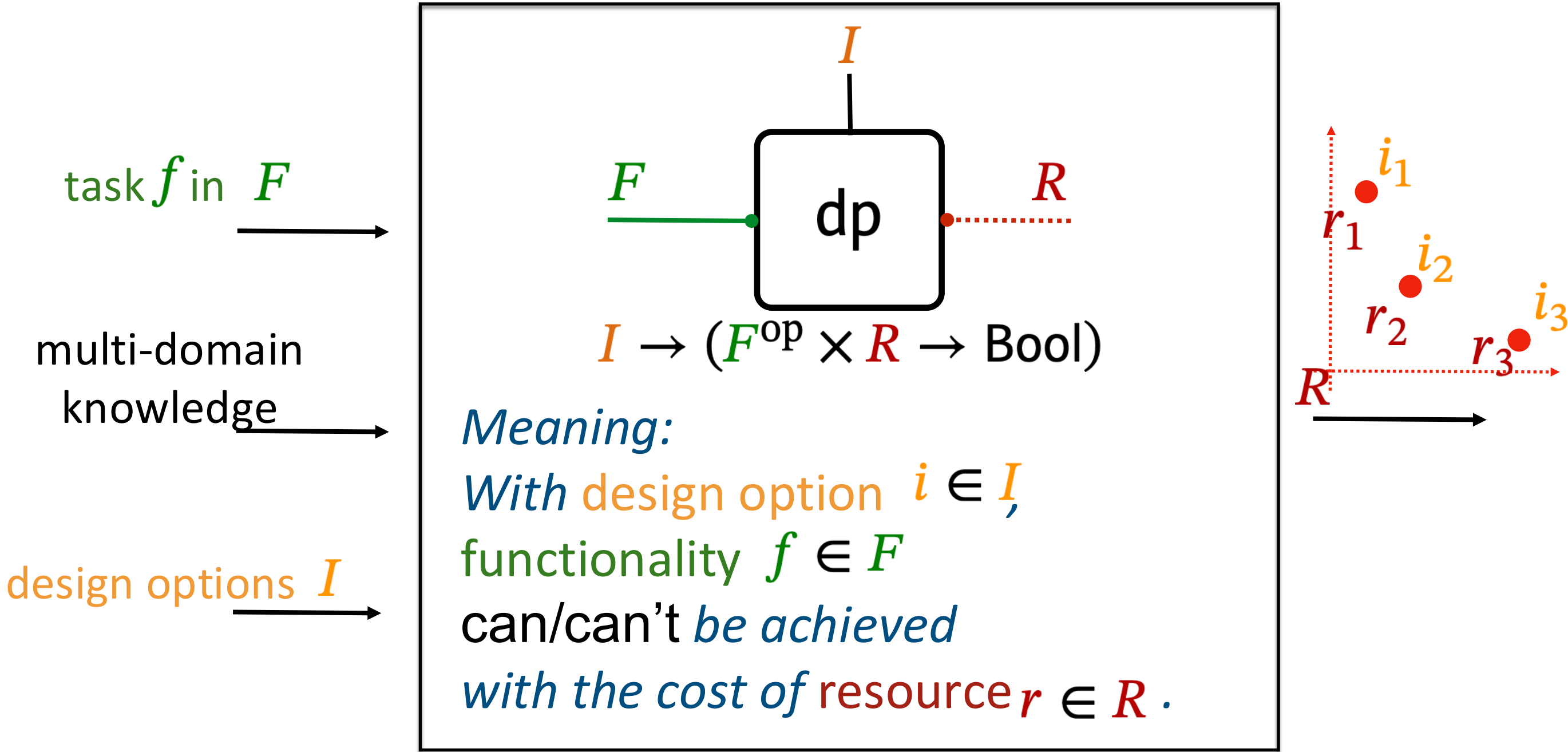
Co-design: abstraction of system design

minimize
resources usage: the resources we need to have available
subject to
functionality constraints: what we need to provide/achieve
with
(combination of)
available design choices: the options we can choose from



Co-design: abstraction of system design

minimize
resources usage: the resources we need to have available
subject to
functionality constraints: what we need to provide/achieve
with
(combination of)
available design choices: the options we can choose from



Your system is just a component in another person's system

Infrastructure level



*Infrastructure choices
and coordination of services*

Service level



Fleet deployment

Platform level



*Choice of components
and algorithms*

Subsystem level



Single component design solutions

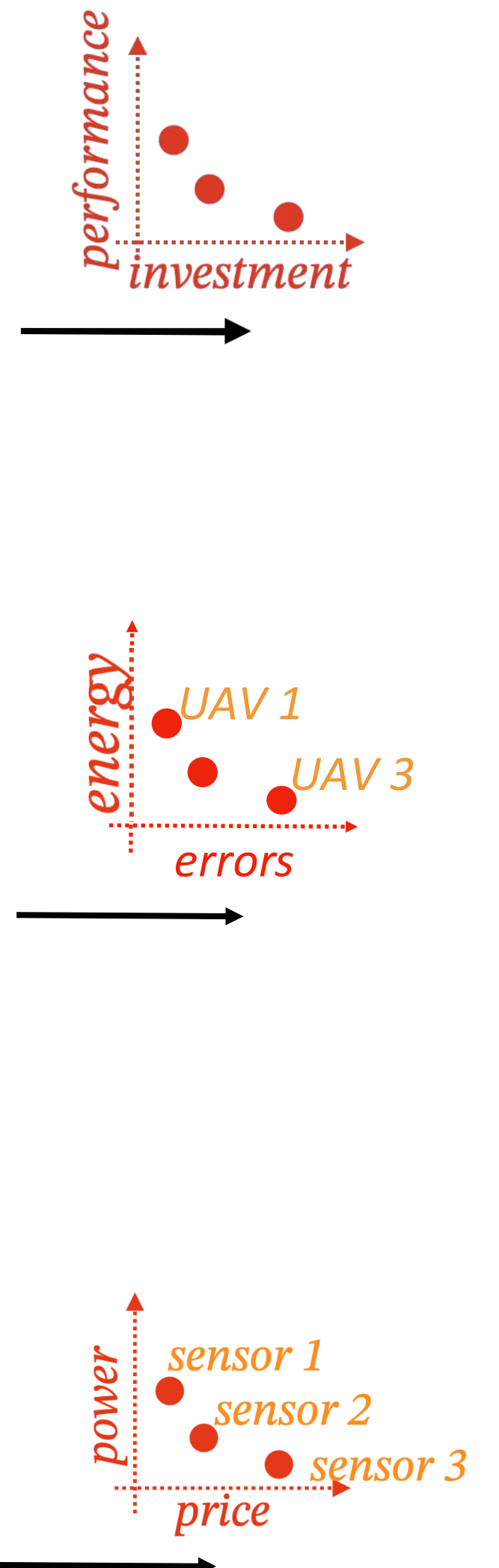
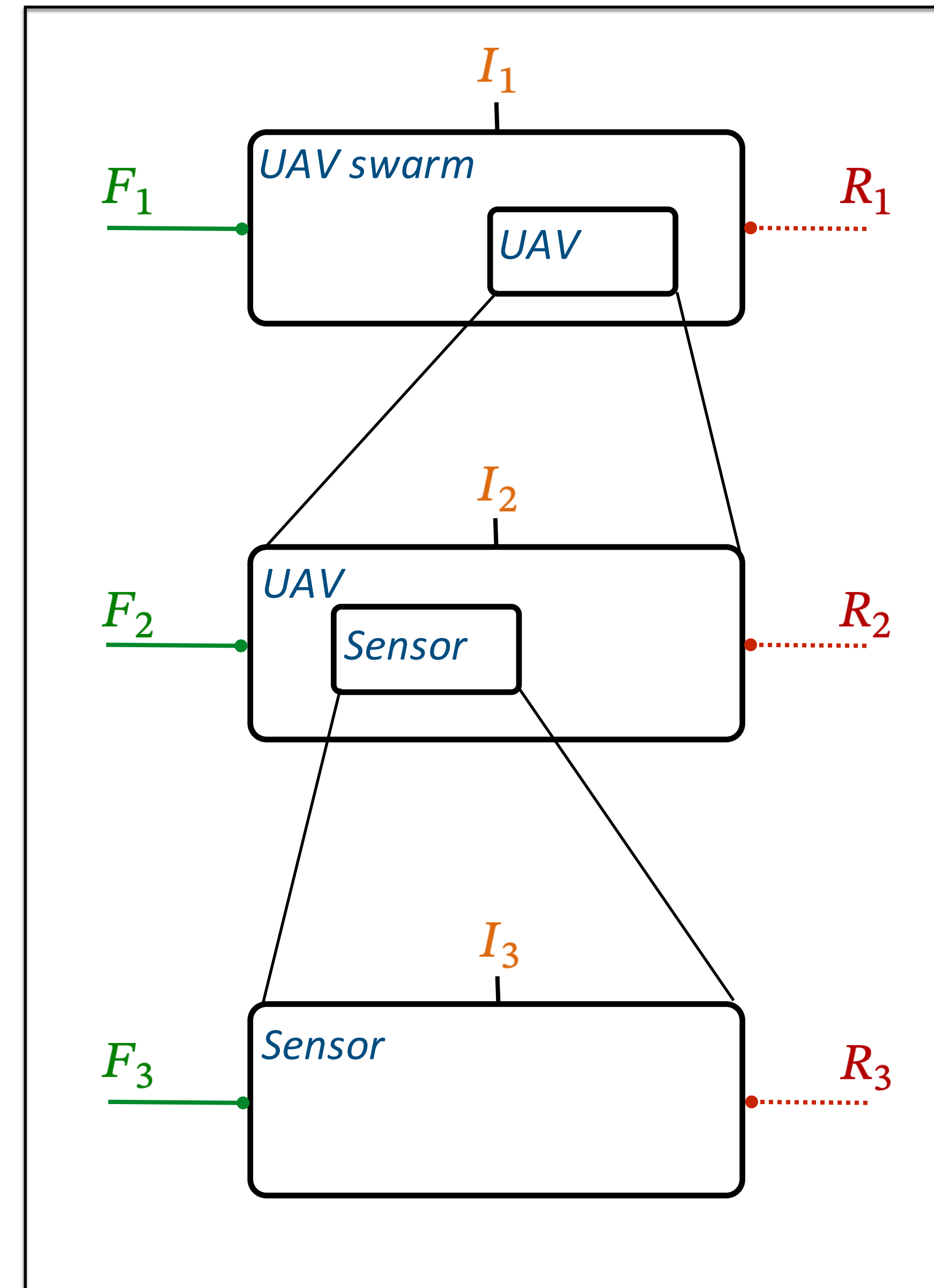
Co-design at scales



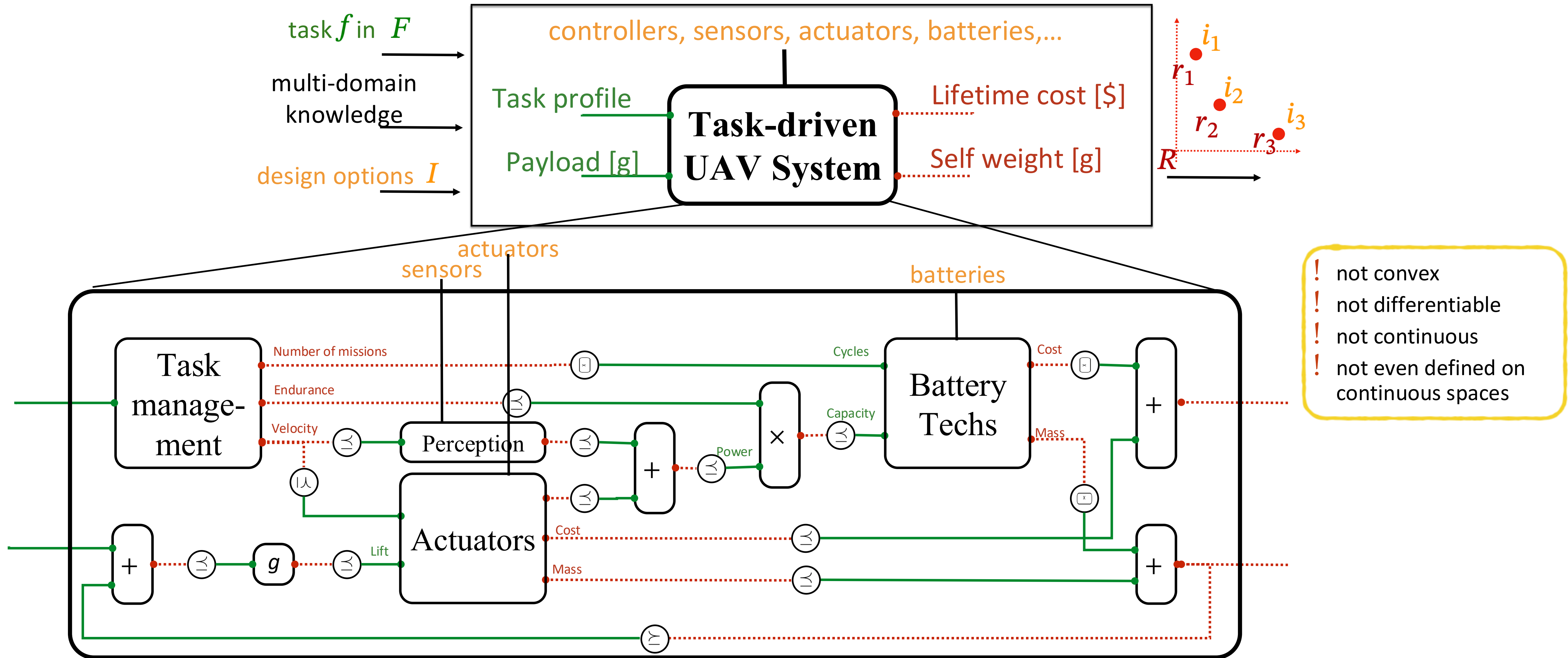
demand
 networks, operations,
 infrastructure, etc.
 mobility services, policies

task profile
 robot autonomy, physics
 components, algorithms

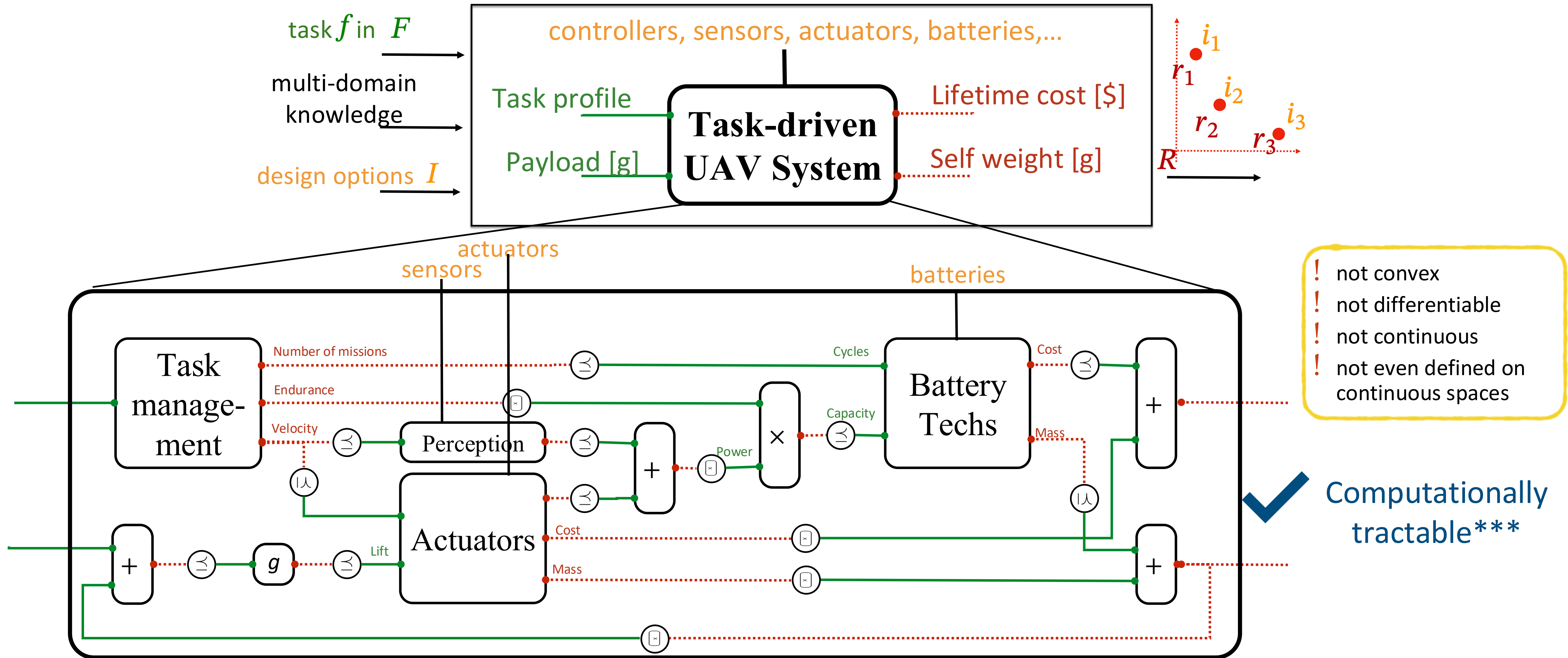
sampling frequency
 physics
 sensor choices



Co-design: abstract, compositional, computational system design



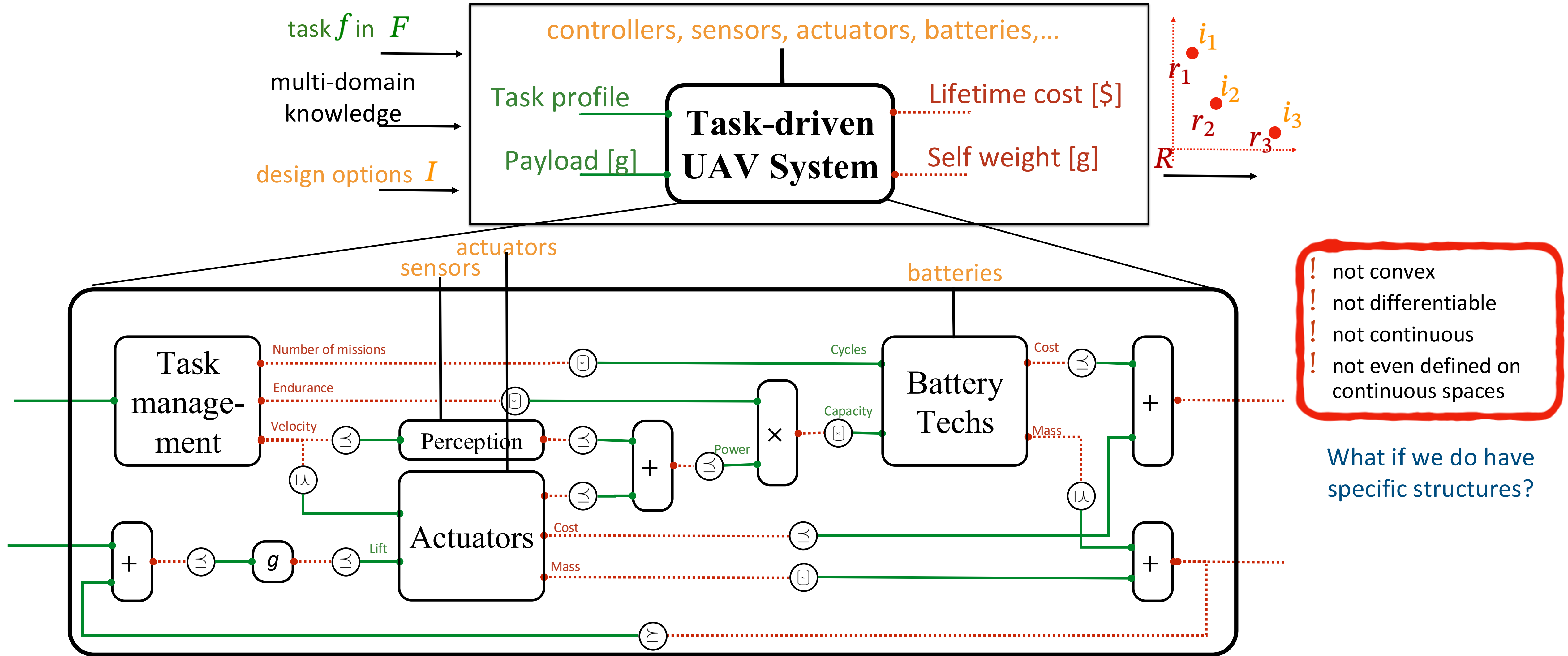
Co-design: abstract, compositional, computational system design



$\text{solution}(\text{composition}(a, b))$

$= \text{composition}(\text{solution}(a), \text{solution}(b))$

Co-design: abstract, compositional, computational system design



Co-design: from the optimization perspective

minimize

resources usage: the resources we need to have available

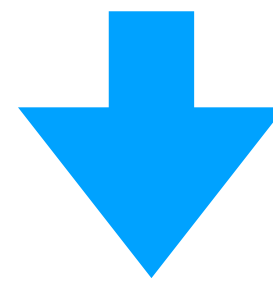
subject to

functionality constraints: what we need to provide/achieve

with

(combination of)

available design choices: the options we can choose from



minimize

resources usage: the resources we need to have available

subject to

functionality constraints: what we need to provide/achieve

Feasibility constraint(s)

$$\begin{array}{ll} \min & x_R \\ \leq_R & \\ \text{s.t.} & \langle \hat{x}_F, x_R \rangle \in \text{dp}(i) \end{array}$$

Abstract away
implementations
(usually discrete)

$$\begin{array}{ll} \min & x_R \\ \leq_R & \\ \text{s.t.} & \hat{x}_F \leq_F x_F \\ & \langle x_F, x_R \rangle \in \text{dp} \end{array}$$

Co-design: from the optimization perspective

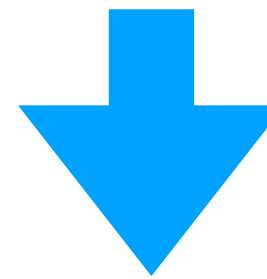
minimize

resources usage: the resources we need to have available

subject to

functionality constraints: what we need to provide/achieve
Feasibility constraint(s)

$$\begin{aligned} \min_{\leq_R} \quad & x_R \\ \text{s.t.} \quad & \hat{x}_F \leq_F x_F \\ & \langle x_F, x_R \rangle \in \text{dp} \end{aligned}$$



Linear constraints

minimize

resources usage: the resources we need to have available

subject to

functionality constraints: what we need to provide/achieve
Linear feasibility constraint(s)

$$\begin{aligned} \min_{\leq_{\mathbb{R}^n}} \quad & x_{\mathbb{R}^n} \\ \text{s.t.} \quad & \hat{x}_{\mathbb{R}^m} \leq_{\mathbb{R}^m} x_{\mathbb{R}^m} \\ & A \begin{bmatrix} x_{\mathbb{R}^m} \\ x_{\mathbb{R}^n} \end{bmatrix} \leq b \end{aligned}$$

Co-design: general definition of Linear design problems

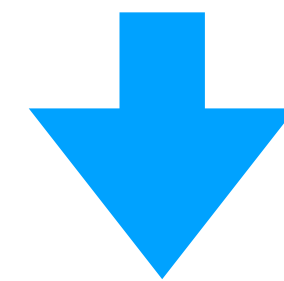
$$\min_{\leq_R} x_R$$

$$\text{s.t. } \hat{x}_F \leq_F x_F$$

$$\langle x_F, x_R \rangle \in \text{dp}$$

Design problem: feasible *functionality*/*resource* pairs:

$$\text{dp} \subseteq F \times R$$



Linear constraints

$$\min_{\leq_{\mathbb{R}^n}} x_{\mathbb{R}^n}$$

$$\text{s.t. } \hat{x}_{\mathbb{R}^m} \leq_{\mathbb{R}^m} x_{\mathbb{R}^m}$$

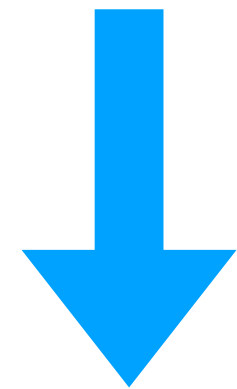
$$A \begin{bmatrix} x_{\mathbb{R}^m} \\ x_{\mathbb{R}^n} \end{bmatrix} \leq b$$

Linear design problem: feasible *functionality*/*resource* pairs as polyhedron

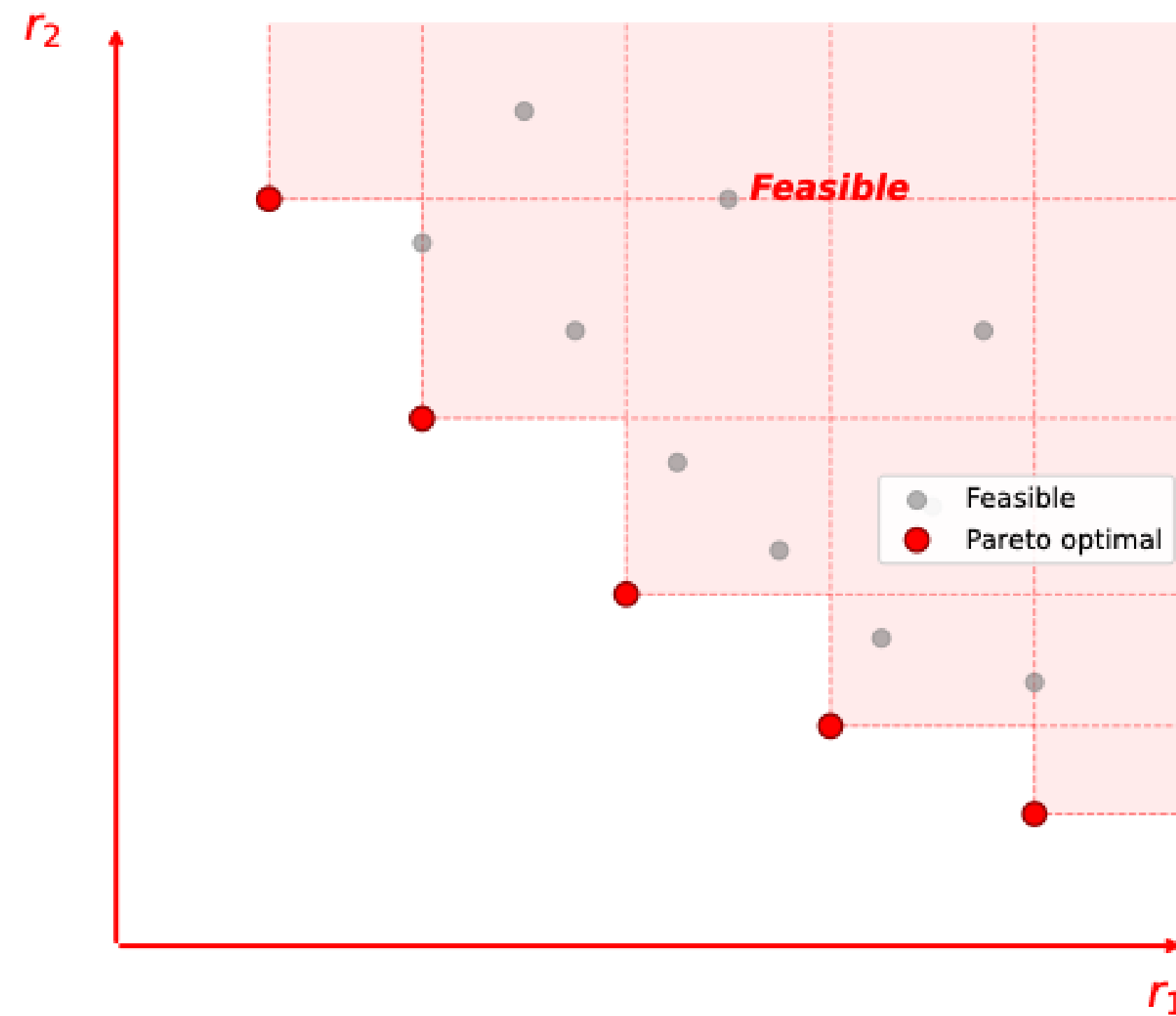
$$\text{ldp} = \left\{ \langle x_{\mathbb{R}^m}, x_{\mathbb{R}^n} \rangle \mid \begin{bmatrix} A_{\mathbb{R}^m} & A_{\mathbb{R}^n} \end{bmatrix} \begin{bmatrix} x_{\mathbb{R}^m} \\ x_{\mathbb{R}^n} \end{bmatrix} \leq b \right\}$$

General DP vs. LDP: Differences in Feasible Sets

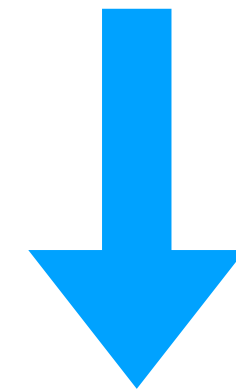
$$\text{dp} \subseteq F \times R$$



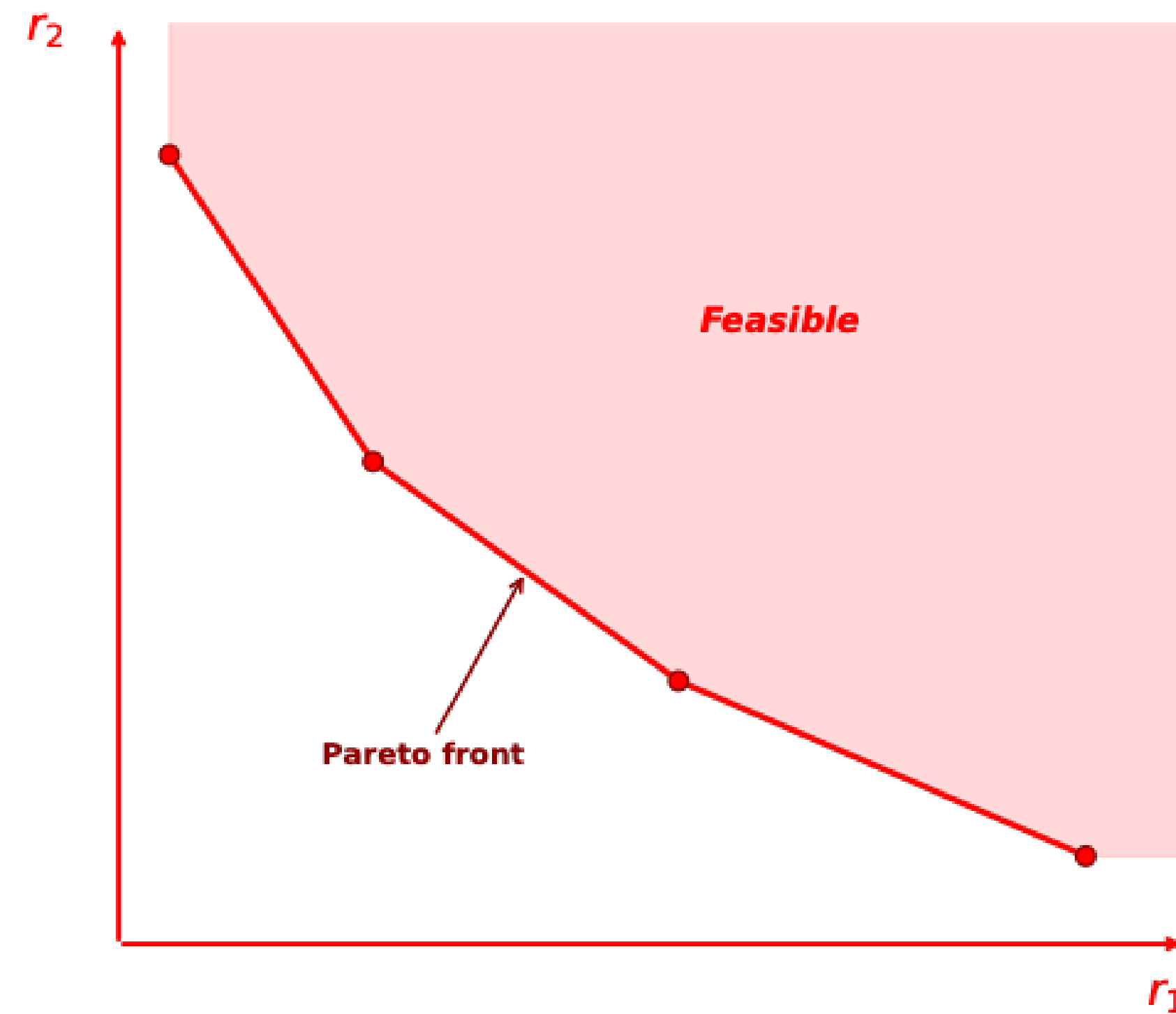
General DP



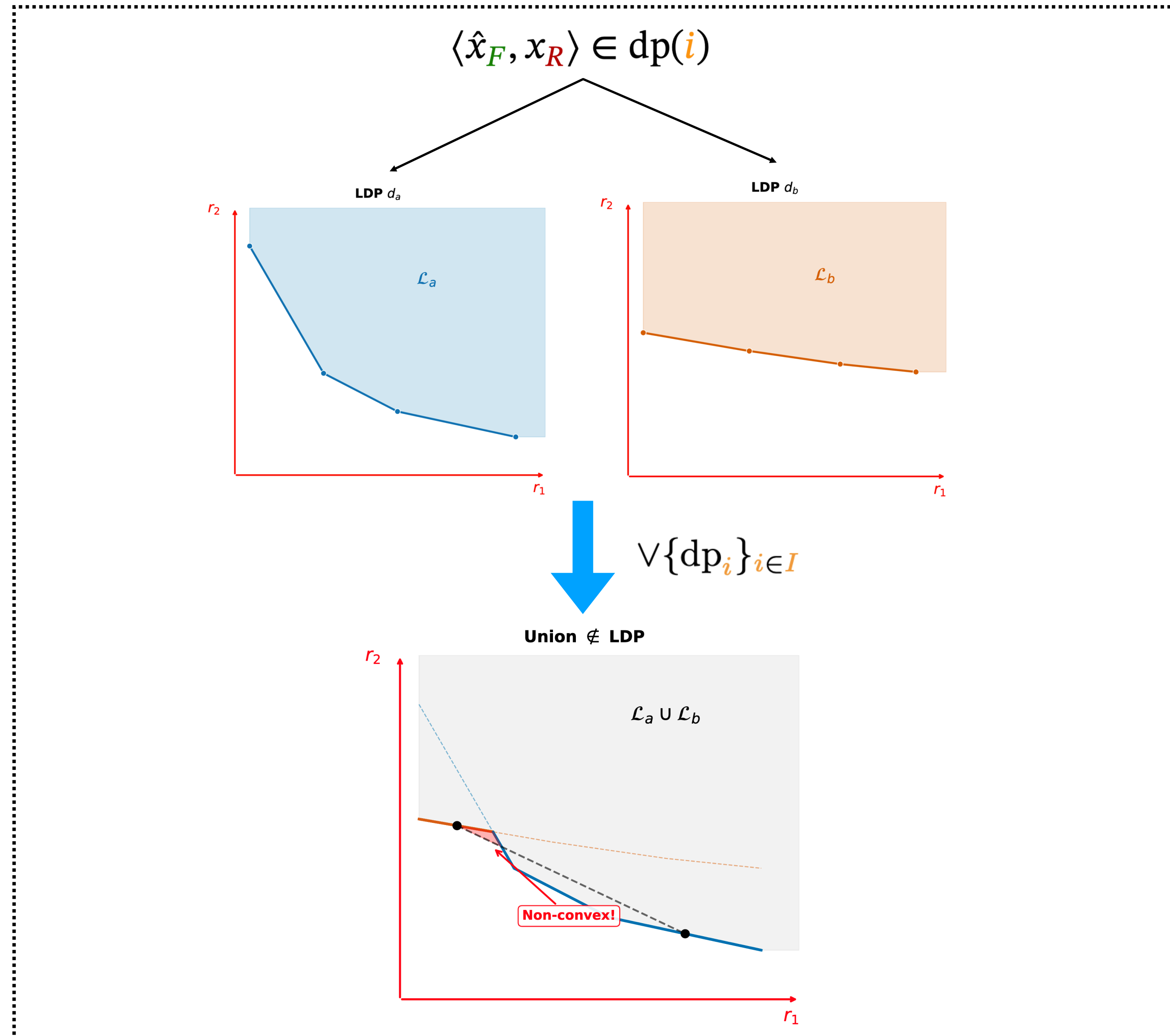
$$\text{ldp} = \left\{ \langle x_{\mathbb{R}^m}, x_{\mathbb{R}^n} \rangle \mid \begin{bmatrix} A_{\mathbb{R}^m} & A_{\mathbb{R}^n} \end{bmatrix} \begin{bmatrix} x_{\mathbb{R}^m} \\ x_{\mathbb{R}^n} \end{bmatrix} \leq b \right\}$$



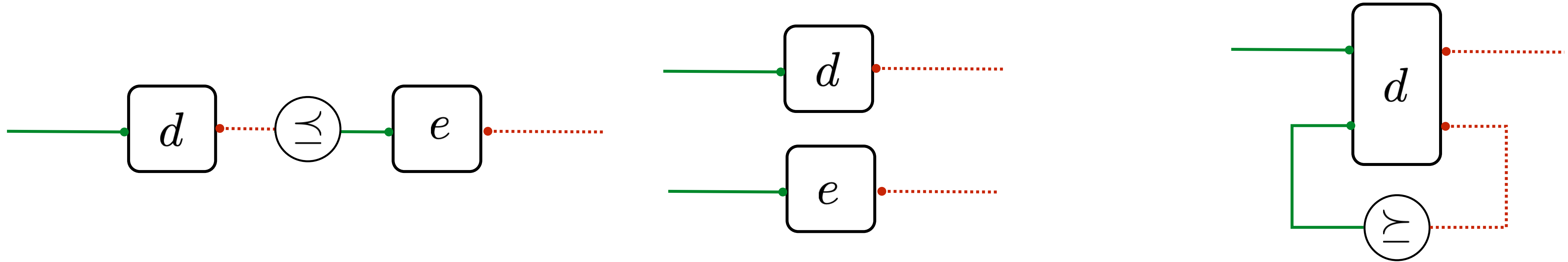
Linear DP



Why not Implementation (Union)

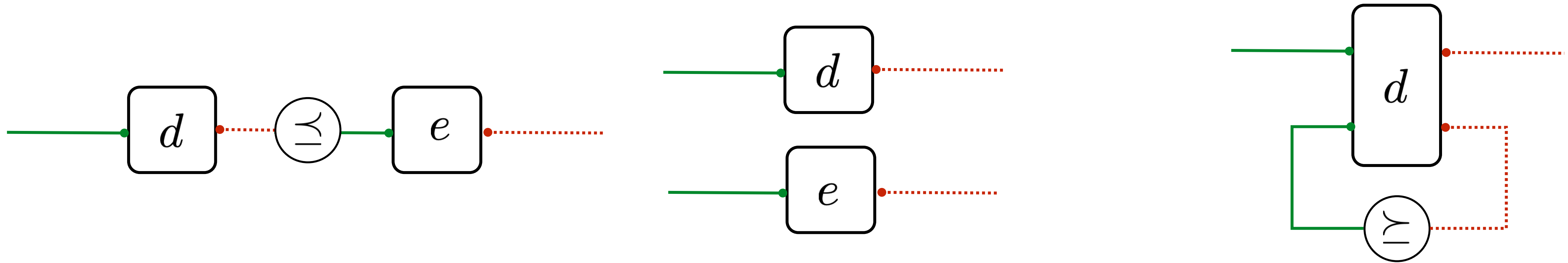


Linear design problems: compositionality

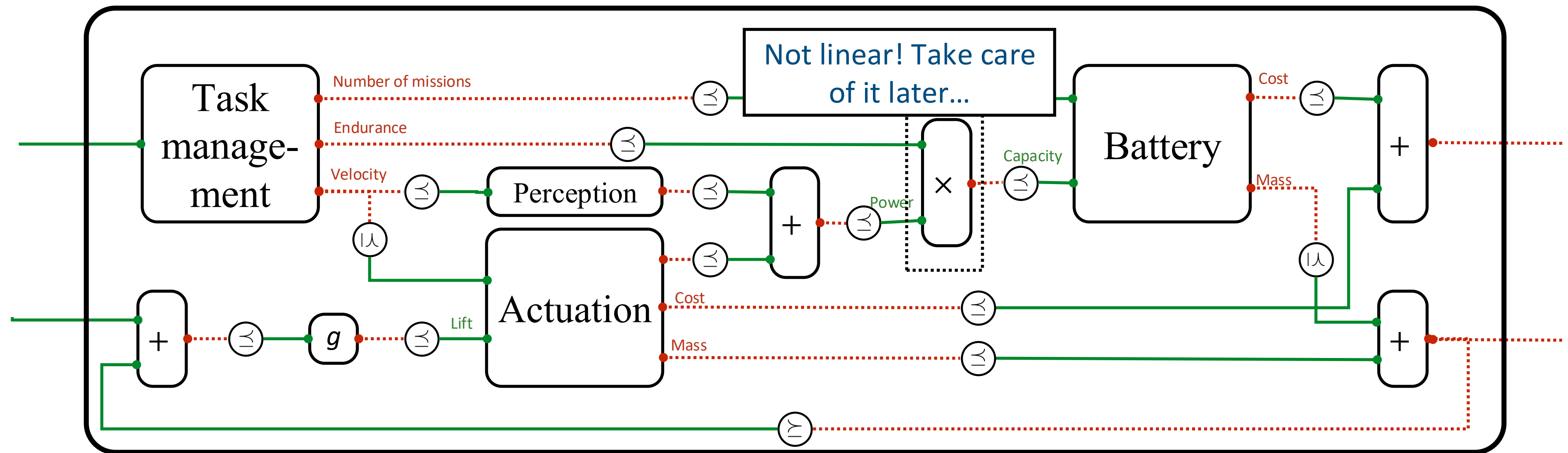


Theorem: LDP is closed under the above compositions

Linear design problems: compositionality

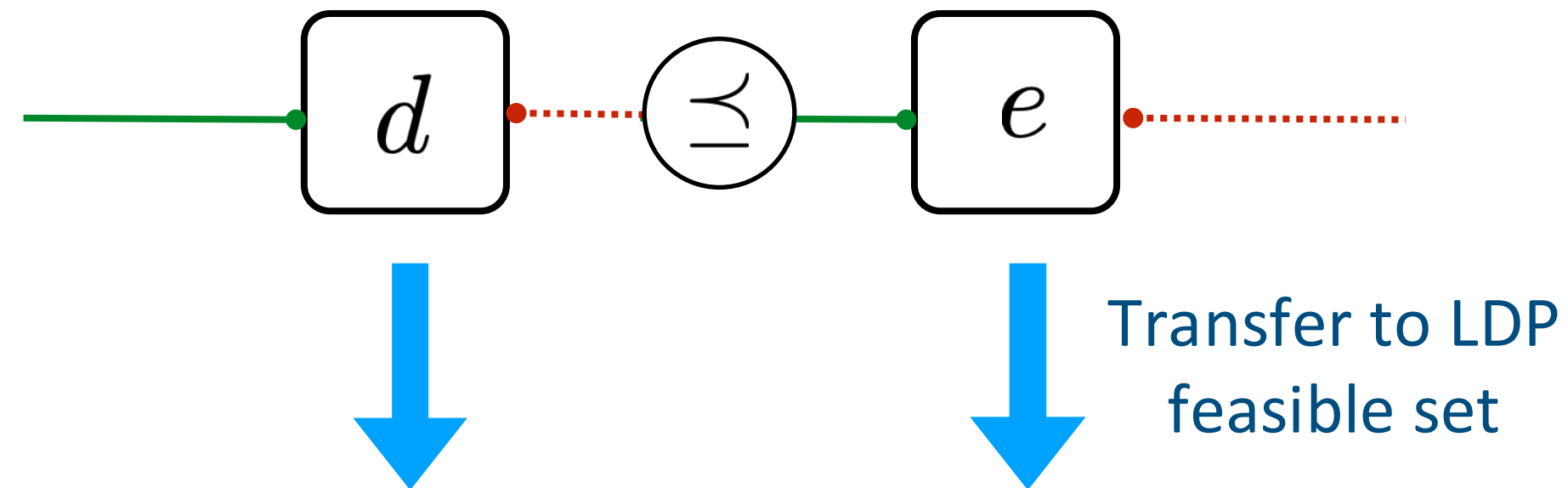


Theorem: LDP is closed under the above compositions



Consequence: One can construct general interconnected LDPs

Closure of LDPs under composition: Series



$$\begin{aligned} \text{ldp}_a &= \{ \langle x_{\mathcal{P}}, x_{\mathcal{Q}} \rangle \mid A_{\mathcal{P}}x_{\mathcal{P}} + A_{\mathcal{Q}}x_{\mathcal{Q}} \geq b_a \} \\ \text{ldp}_b &= \{ \langle x_{\mathcal{Q}}, x_{\mathcal{R}} \rangle \mid B_{\mathcal{Q}}x_{\mathcal{Q}} + B_{\mathcal{R}}x_{\mathcal{R}} \geq b_b \} \end{aligned}$$

Def of Series

$$\{ \langle x_{\mathcal{P}}, x_{\mathcal{R}} \rangle \mid \exists x_{\mathcal{Q}} : \langle x_{\mathcal{P}}, x_{\mathcal{Q}} \rangle \in \text{dp}_a \text{ and } \langle x_{\mathcal{Q}}, x_{\mathcal{R}} \rangle \in \text{dp}_b \}$$

Decode in LDP

$$\text{ldp}_{\text{series}} = \{ \langle x_{\mathcal{P}}, x_{\mathcal{R}} \rangle \mid \exists x_{\mathcal{Q}} : A_{\mathcal{P}}x_{\mathcal{P}} + A_{\mathcal{Q}}x_{\mathcal{Q}} \geq b_a, \\ B_{\mathcal{Q}}x_{\mathcal{Q}} + B_{\mathcal{R}}x_{\mathcal{R}} \geq b_b \}$$

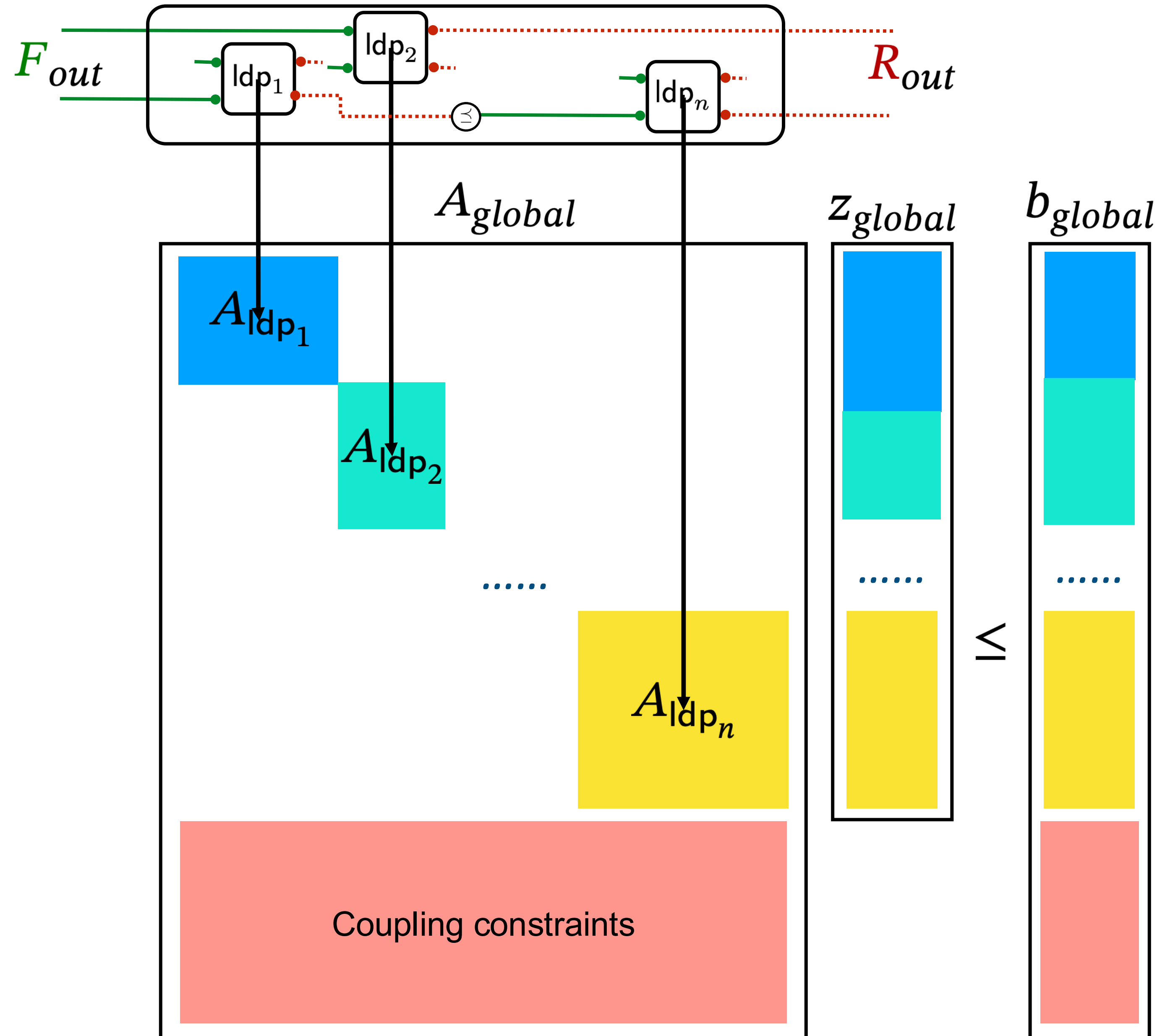
reformulate

$$\begin{bmatrix} A_{\mathcal{P}} & A_{\mathcal{Q}} & 0 \\ 0 & B_{\mathcal{Q}} & B_{\mathcal{R}} \end{bmatrix} \begin{bmatrix} x_{\mathcal{P}} \\ x_{\mathcal{Q}} \\ x_{\mathcal{R}} \end{bmatrix} \geq \begin{bmatrix} b_a \\ b_b \end{bmatrix}$$

$\text{ldp}_{\text{series}}$ is its projection onto the $\langle x_{\mathcal{P}}, x_{\mathcal{R}} \rangle$ coordinates

Similar to parallel, intersection, and feedback!

System query as monolithic optimization problem



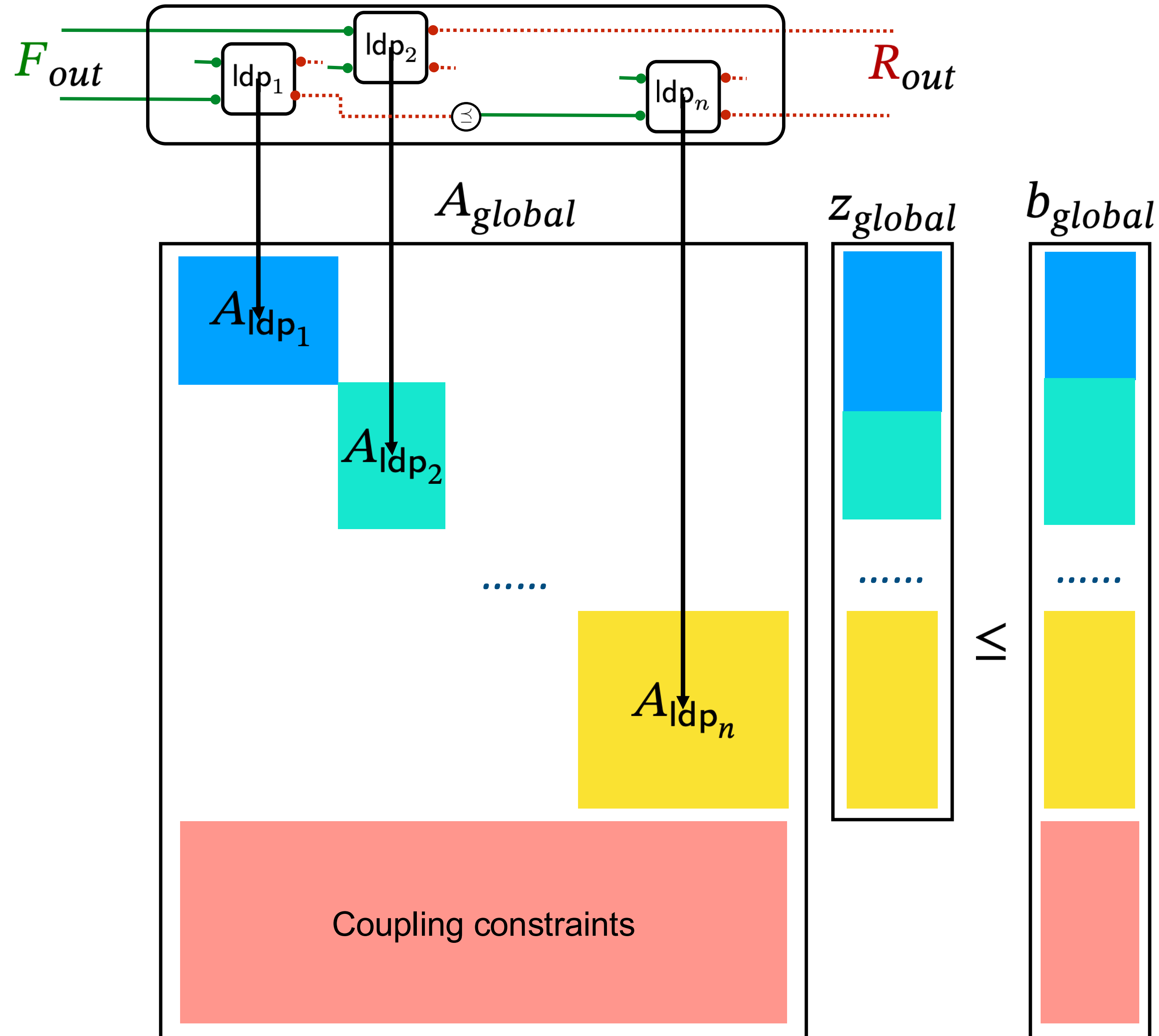
$$\begin{aligned}
 & \min_{\preceq_{R_{out}}} x_{R_{out}} \\
 & \text{s.t.} \quad \hat{x}_{F_{out}} \preceq_{F_{out}} x_{F_{out}} \\
 & \quad A_{global} z_{global} \leq b_{global}
 \end{aligned}$$

Coupling constraint

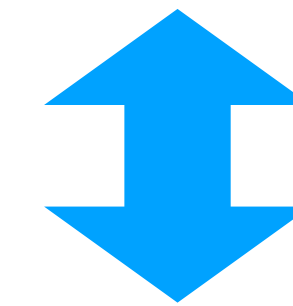
$$x_R \preceq x_R \quad \text{or} \quad x_R = x_R$$

We can use equality due to monotonicity!

System query as monolithic optimization problem



$$\begin{aligned} \min \quad & x_{R_{out}} \\ \text{s.t.} \quad & \hat{x}_{F_{out}} \leq_{F_{out}} x_{F_{out}} \\ & A_{global} z_{global} \leq b_{global} \end{aligned}$$



Multi-objective Linear Programming

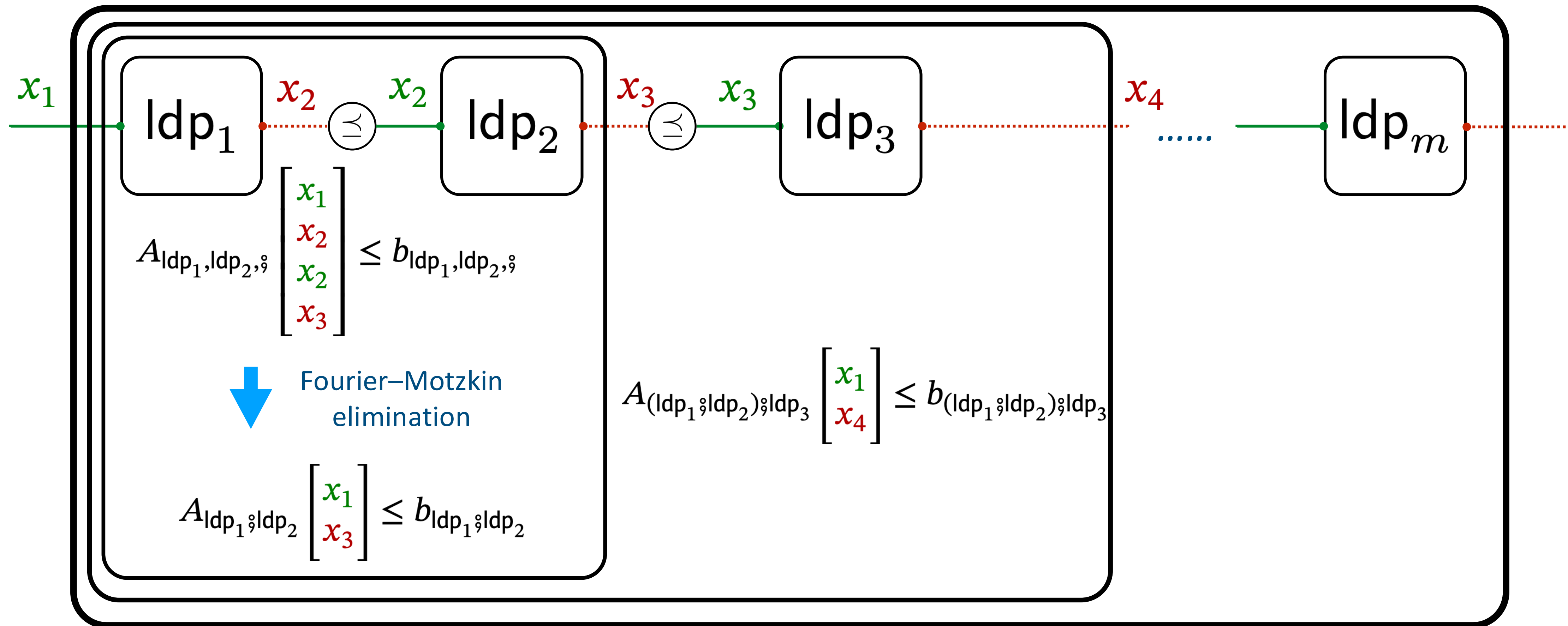
Polyhedron projection

$$\begin{aligned} \min \quad & x_{R_{out}} \\ \text{s.t.} \quad & \langle \hat{x}_{F_{out}}, x_{R_{out}} \rangle \in \text{proj}(\{z_{global} \mid A_{global} z_{global} \leq b_{global}\}) \end{aligned}$$

There's efficient solver!

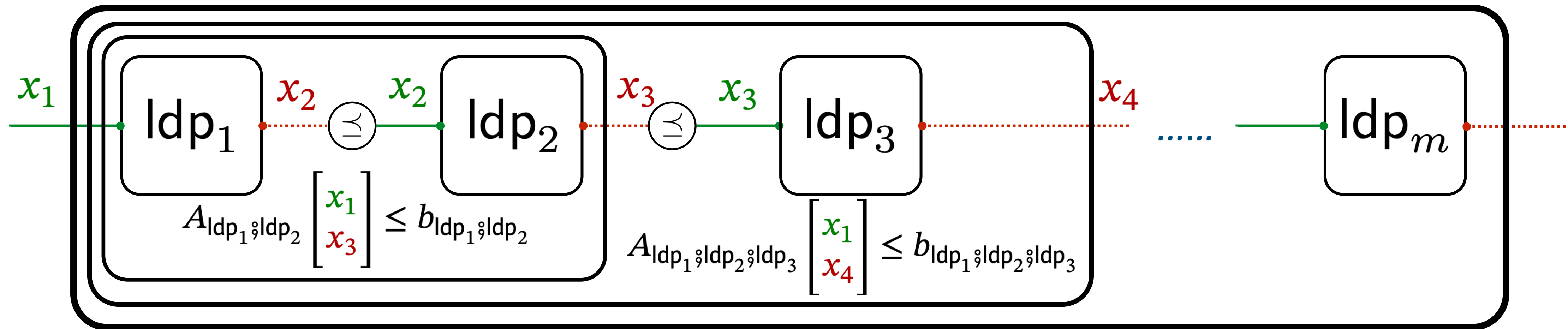
[1] A. Löhne and B. Weißing, "The vector linear program solver *Bensolve* – notes on theoretical background," *European Journal of Operational Research*, vol. 260, no. 3, pp. 807–813, Aug. 2017.

Compositional computation of LDP



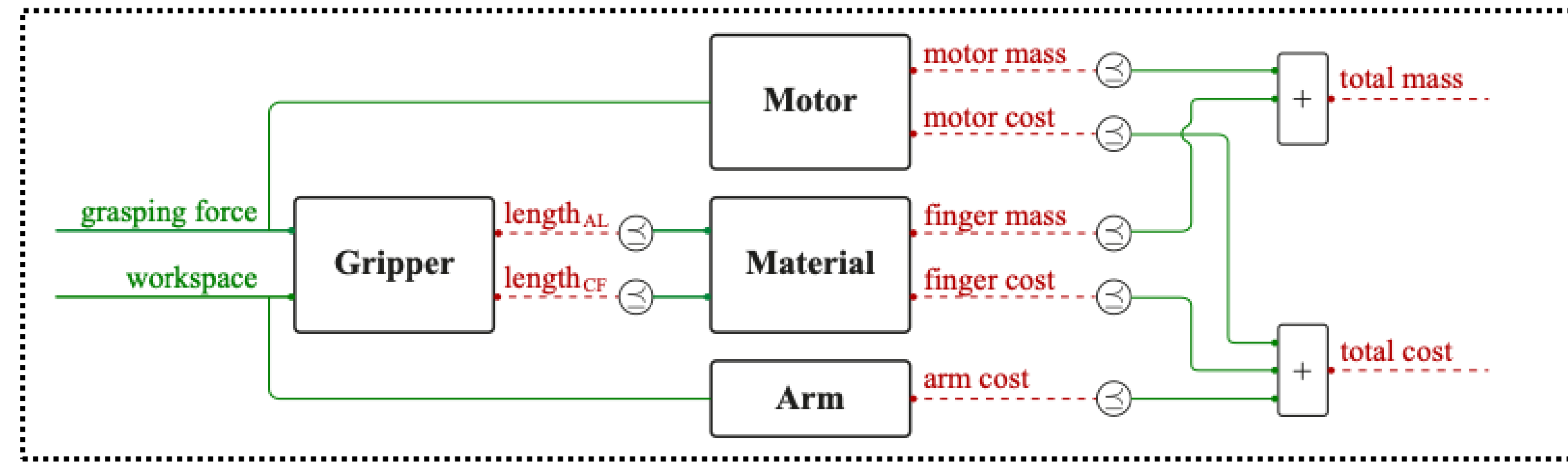
- Pros and cons:
 - Dimension of variables always small
 - Good for repeated queries on the same system/re-computation with few component changes
 - Good for computation when only several components frequently changes
 - Number of constraints might be exponential!

Compositional computation of LDP

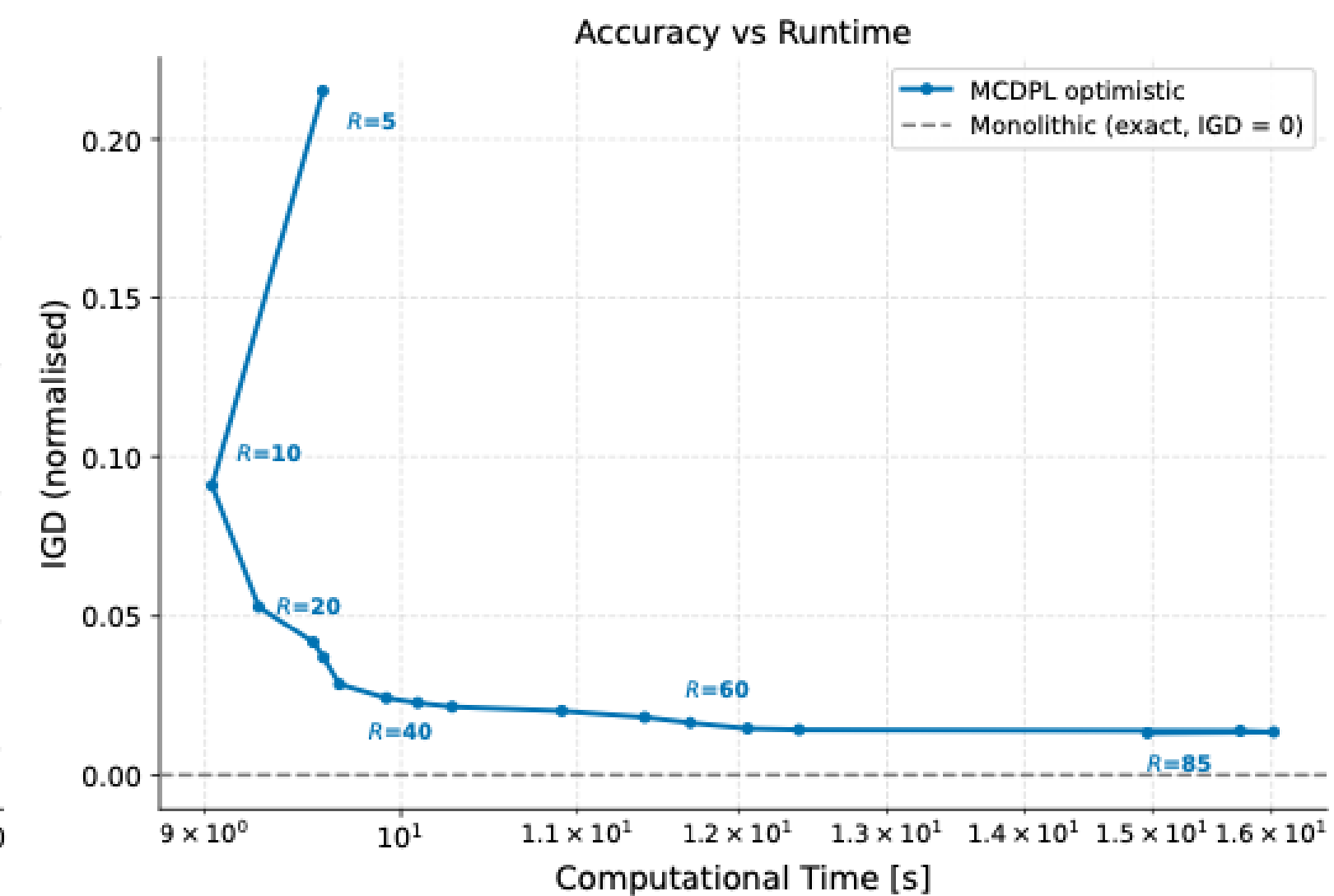
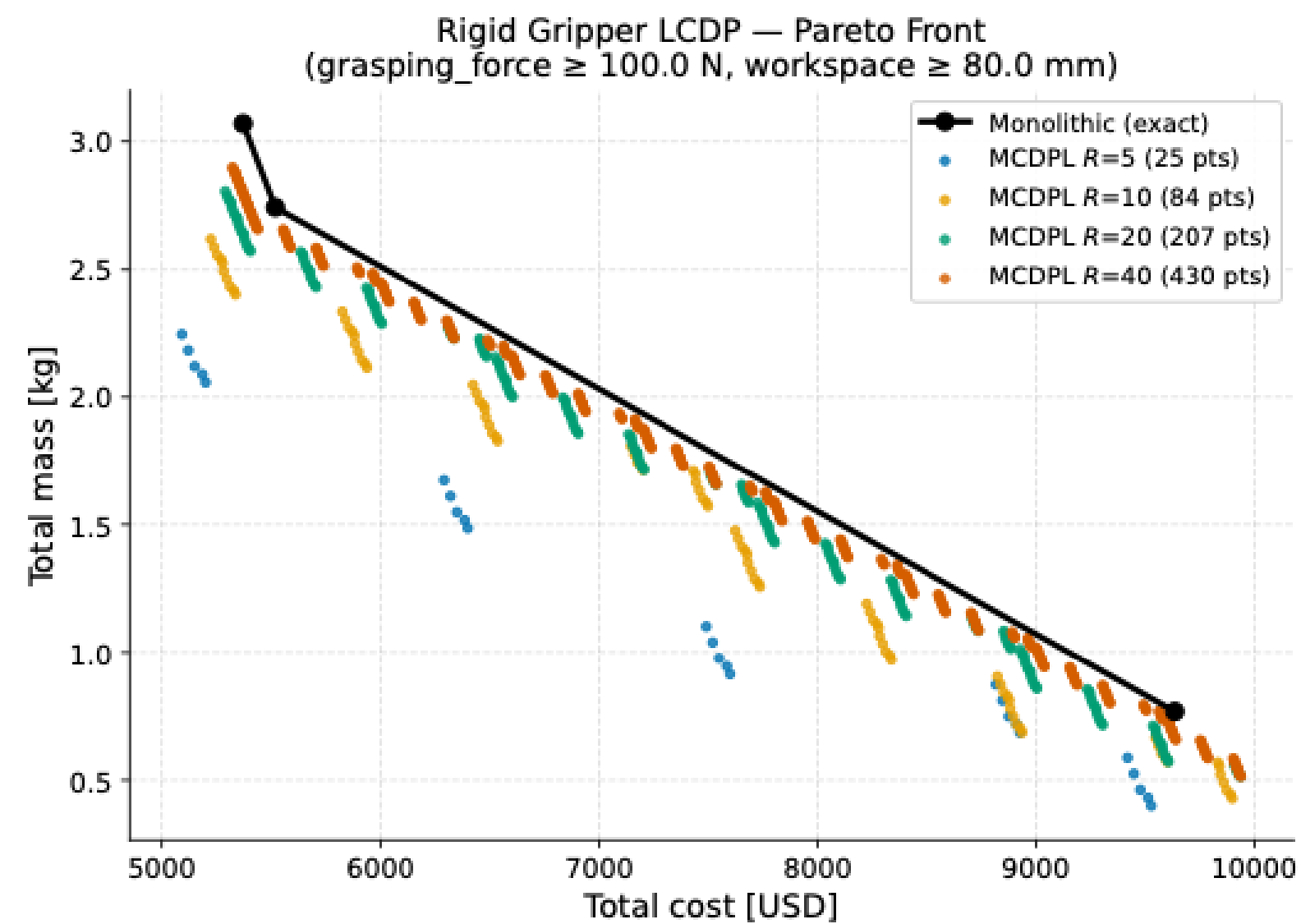


m	Monolithic			Hybrid FME			MCDPL		Agree
	Vars	Cstr	Time	Vars	Cstr	Time	Time	Pts	
3	12	17	0.018	4	12	0.328	11.1	27	✓
4	16	24	0.011	4	14	0.584	11.2	34	✓
6	24	38	0.010	4	26	1.680	47.2	39	✓
8	32	52	0.024	4	20	3.083	19.0	22	✓
10	40	66	0.020	4	29	5.598	27.0	44	✓
15	60	101	0.031	4	36	31.697	43.5	44	✓
20	80	136	0.044	4	40	61.240	67.3	44	✓
30	120	206	0.090	64	151	33.612	127.0	44	✓

Case Study: Monolithic vs MCDPL



All components are LDP!



(ε, δ) -Approximation Theorem

For any convex feasible set and $\varepsilon > 0$, there exists a polyhedral LDP that is an **ε -accurate outer approximation**.

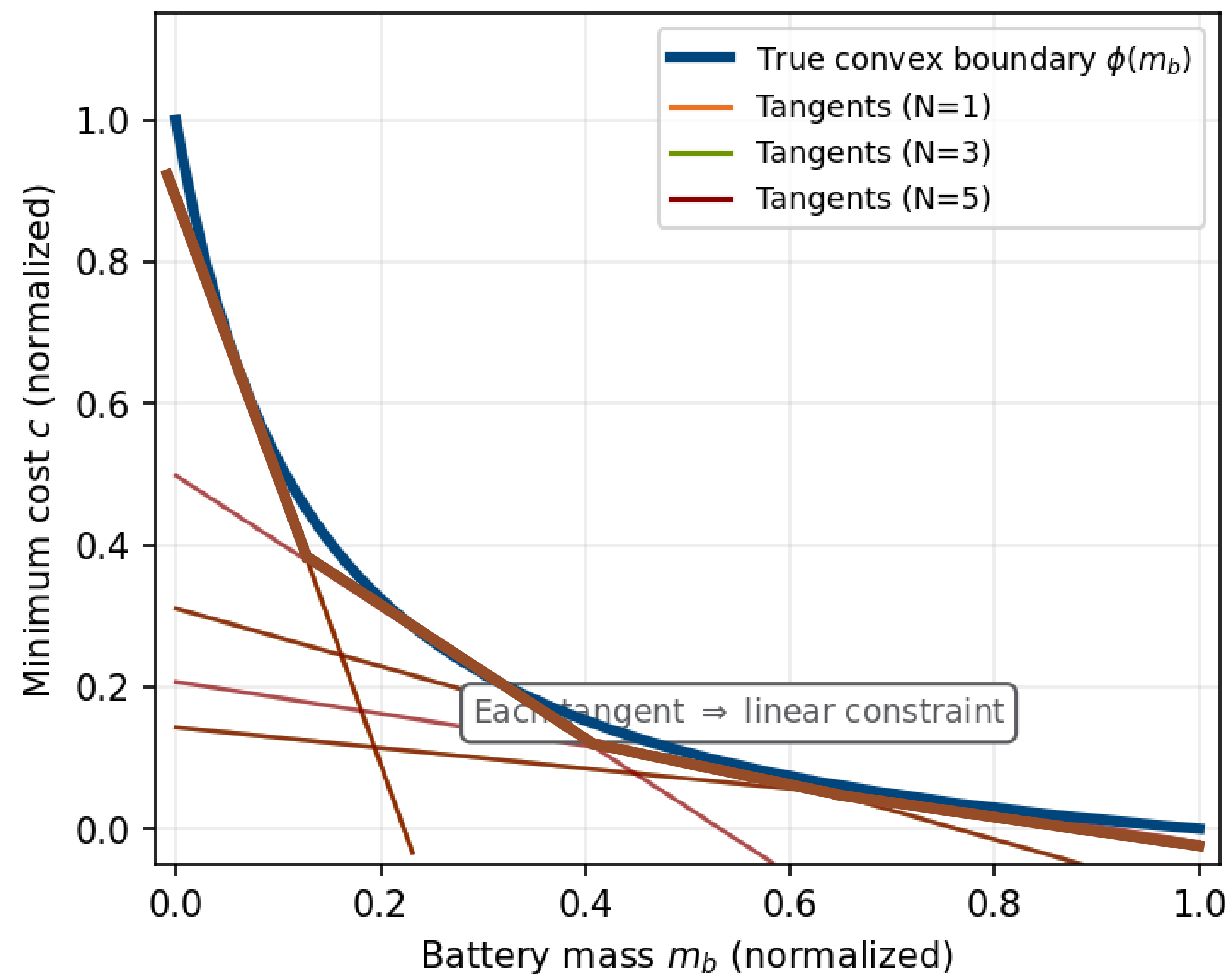
Key: $\delta = 0$ is *structurally guaranteed* for standard resource cones.

$\Rightarrow$ No coverage gap — the approximation never *misses* part of the true frontier.

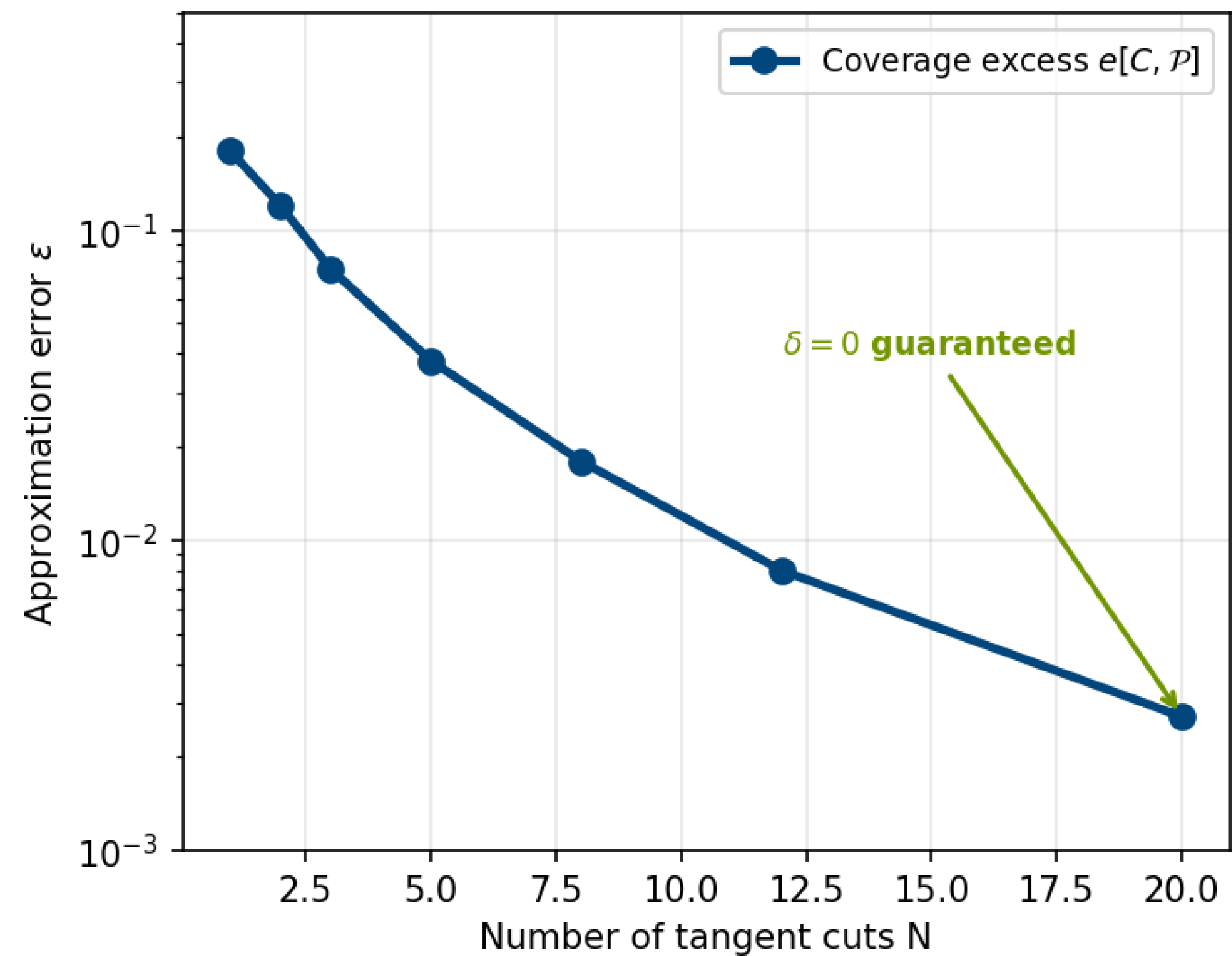
- Practical strategy: tangent-plane linearization
 - Select N anchor points on the convex boundary
 - Add one affine constraint per anchor (tangent $\rightarrow$ linear inequality)
 - Result: an LDP that tightens as N increases

Outer approximation of convex sets

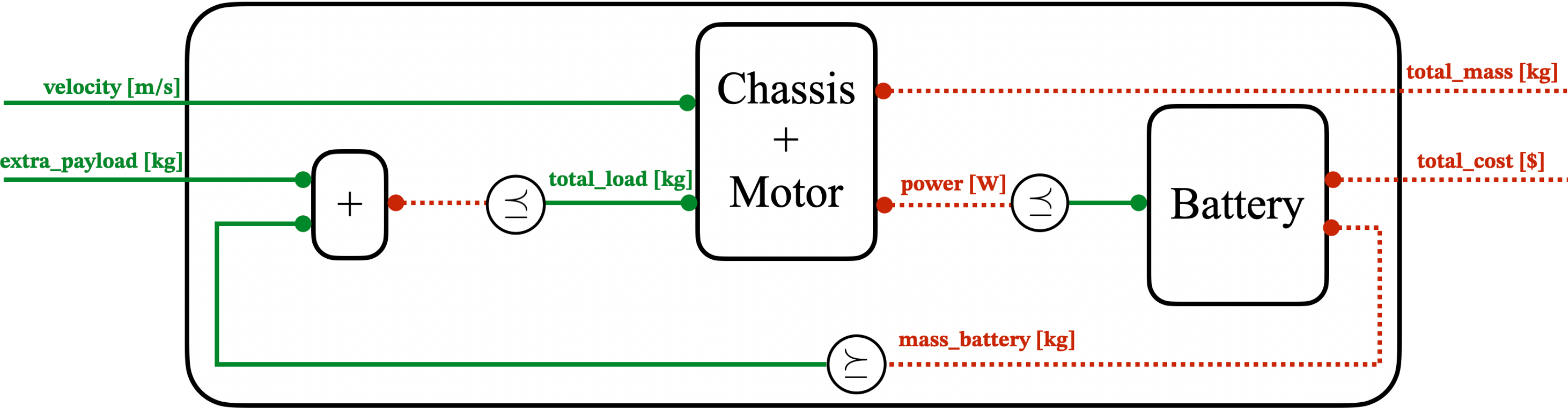
Tangent-plane Outer Approximation of Convex Boundary



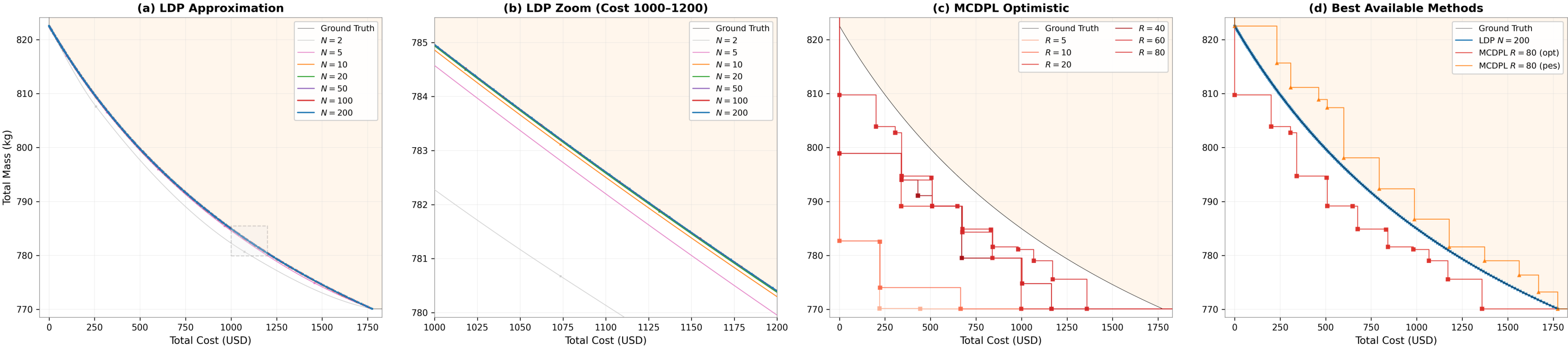
$(\varepsilon, 0)$ -Convergence Guarantee as N increases



Example: rover and battery with bi-linear constraints



$$p_{\text{bat}} \leq \alpha m_b + \beta c + \gamma \underbrace{m_b c}_{\text{bilinear!}}$$

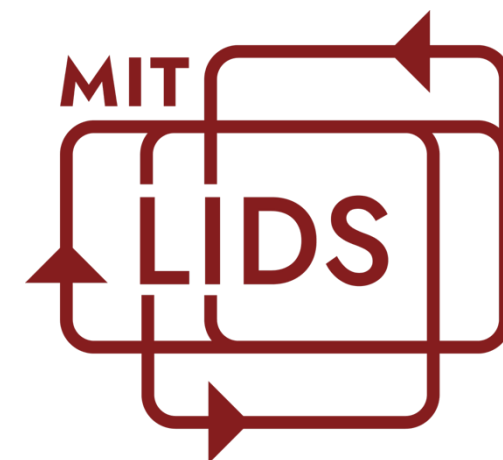


Scalable Co-Design via Linear Design Problems: Theory and Decomposition Algorithm

- ▶ Co-design: layer-wise optimal system design with (deterministic) guarantees
- ▶ Linear co-design: special structure in the problem
 - Compositional: preserved by composing
 - Gives more efficient algorithm
 - Allows approximation of non-linear sets
- ▶ Native compositional Multi-objective Linear Programming Solver?
 - Utilize the topology of composing graph in multi-obj linear programming solver?
- ▶ Warm starter of the general algorithm?
- ▶ Compositional convex programming?
- ▶ Uncertainty and compositional stochastic multi-objective linear programming?

Scalable Co-Design via Linear Design Problems: Theory and Decomposition Algorithm

Sponsored by DARPA Young Faculty Award (YFA) program



More on co-design

